BÀI 10
API

A. MỤC TIÊU:

- Kết nối ứng dụng Flutter với Web API .NET Core
- Thực hiện GET/POST/PUT/DELETE
- Triển khai đăng nhập và truy cập API protected bằng JWT.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

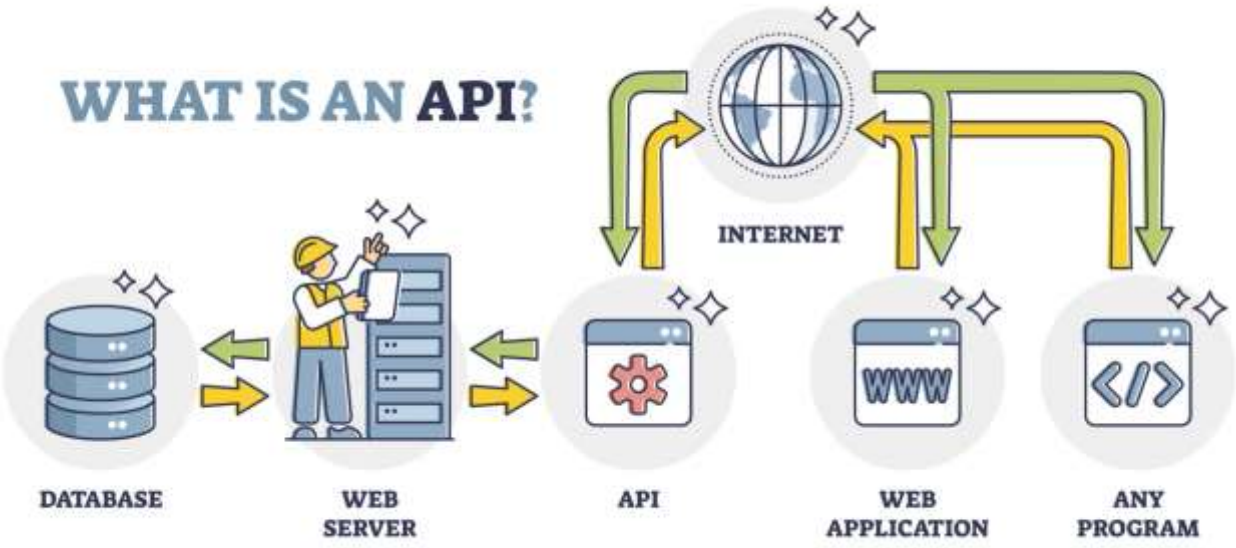
STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

a. Khái niệu về API

➢ API là viết tắt của Application Programming Interface (giao diện lập trình ứng dụng), là API là "cầu nối" giúp các phần mềm khác nhau nói chuyện và làm việc với nhau.



➢ Có nhiều loại API như:

- REST (Representational State Transfer): Chúng có đặc điểm là dựa dựa trên giao thức HTTP, sử dụng JSON làm định dạng dữ liệu chính. Ưu điểm: Nhẹ, linh hoạt, dễ học và tương thích tốt với các ứng dụng Web/Mobile.

- SOAP (Simple Object Access Protocol): Đặc điểm: Sử dụng định dạng dữ liệu XML và có quy định rất khắt khe về bảo mật, cấu trúc. Ưu điểm: Tính bảo mật cực cao, thường dùng trong các hệ thống ngân hàng hoặc giao dịch tài chính lớn.
- GraphQL: Được phát triển bởi Facebook, cho phép người dùng truy vấn đúng những gì họ cần (không thừa, không thiếu). Ưu điểm: Hiệu quả hơn REST khi cần lấy dữ liệu phức tạp từ nhiều nguồn khác nhau.
- gRPC (Google Remote Procedure Call): Sử dụng giao thức HTTP/2 và định dạng nhị phân (Protocol Buffers) thay vì văn bản. Uu điểm: Tốc độ cực nhanh, thường dùng để kết nối các Microservices trong hệ thống lớn

➢ Kiến trúc kết nối

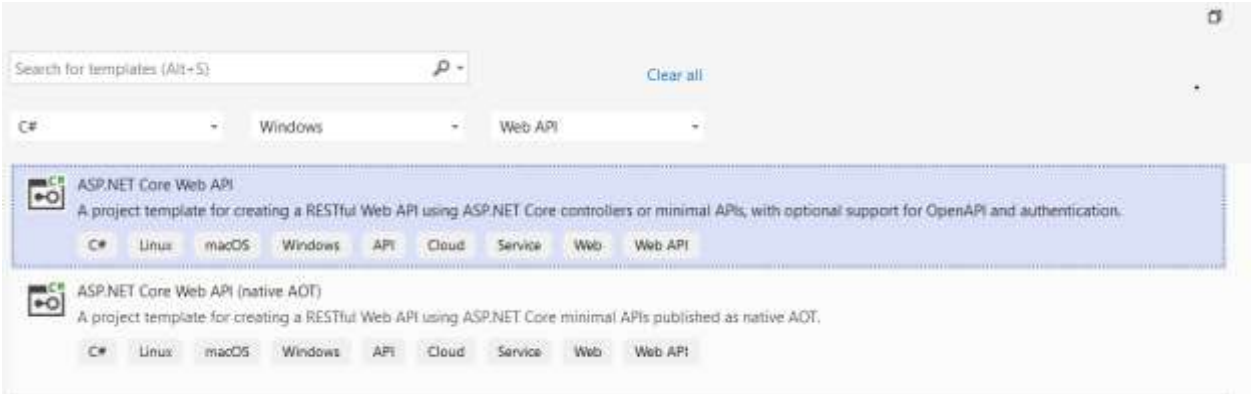


➢ Trong bài học này, chúng ta sẽ sử REST API để có thể kết nối dữ liệu. Có 4 phương thức Http thông dụng như sau:

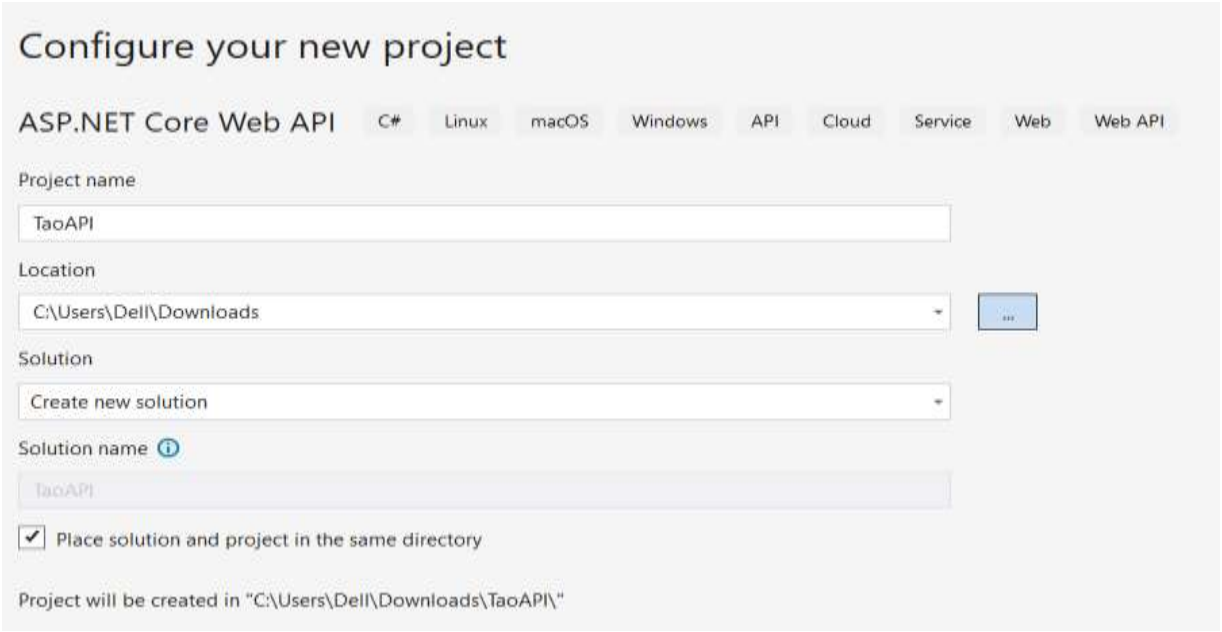
Phương thức	Vai trò	Gợi ý triển khai	Kết quả
GET	Lấy dữ liệu từ server	Dùng để lấy danh sách sản phẩm hoặc lấy chi tiết theo id	JSON danh sách hoặc object
POST	Tạo mới dữ liệu	Gửi body JSON từ Flutter lên API	Bản ghi mới được tạo
PUT	Cập nhật toàn bộ dữ liệu	Gửi object cập nhật lên endpoint /{id}	Đối tượng sau cập nhật
DELETE	Xóa dữ liệu	Gửi request xóa theo id	Thông báo xóa thành công

b. Xây dựng project với NetCore và EntityFramework (với Microsoft Visual studio 2022)

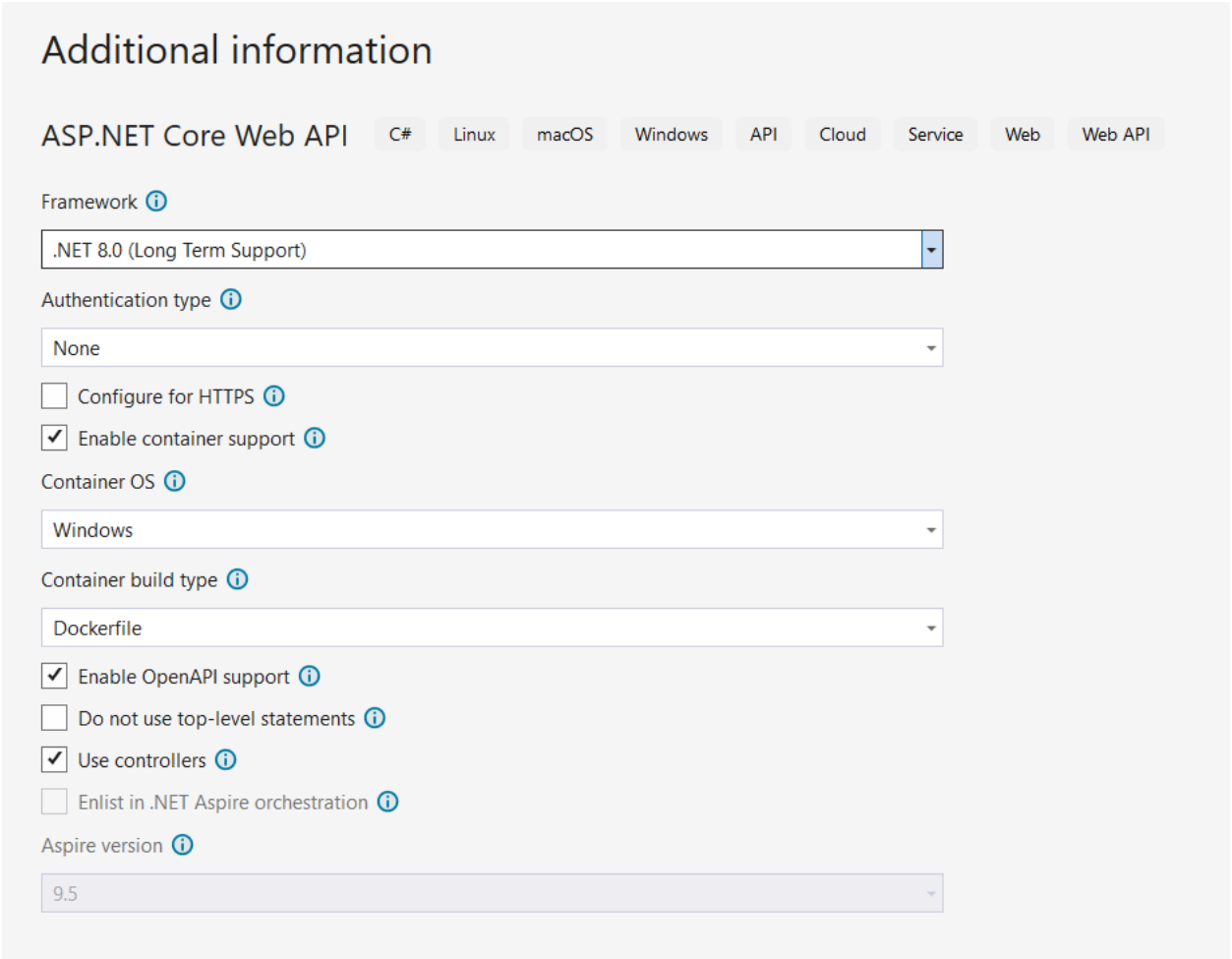
➢ Khởi tạo project: Chọn loại project ASP.NET Core Web API



➢ Chọn vị trí lưu trữ

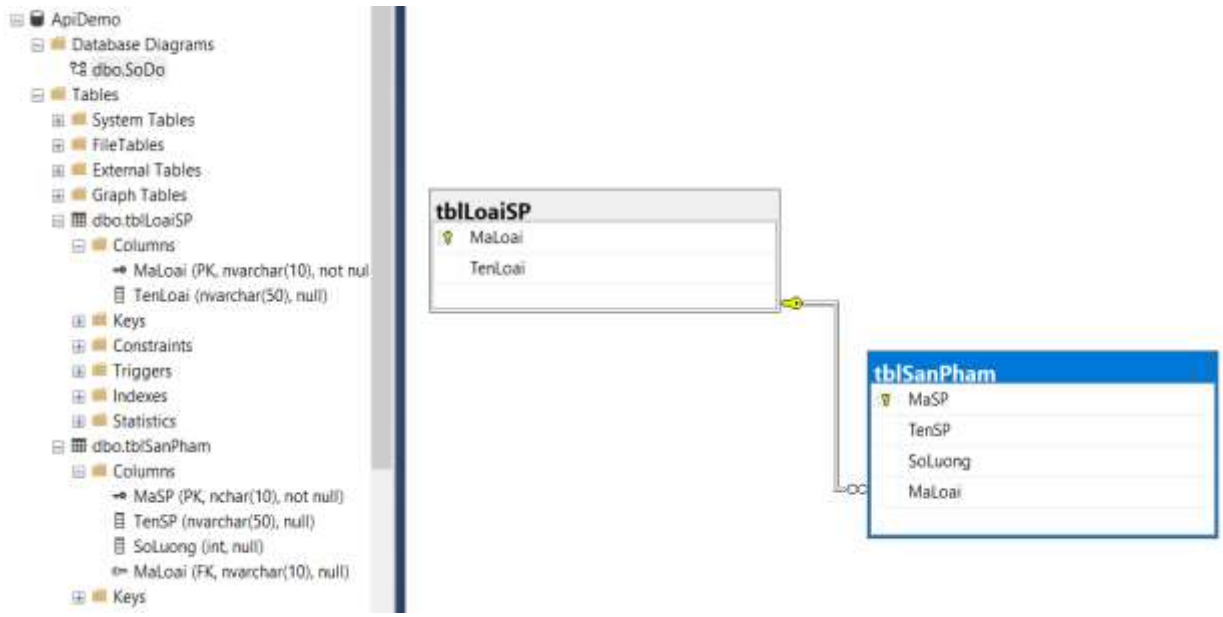


➢ Cài đặt một số thông số ban đầu cho web API



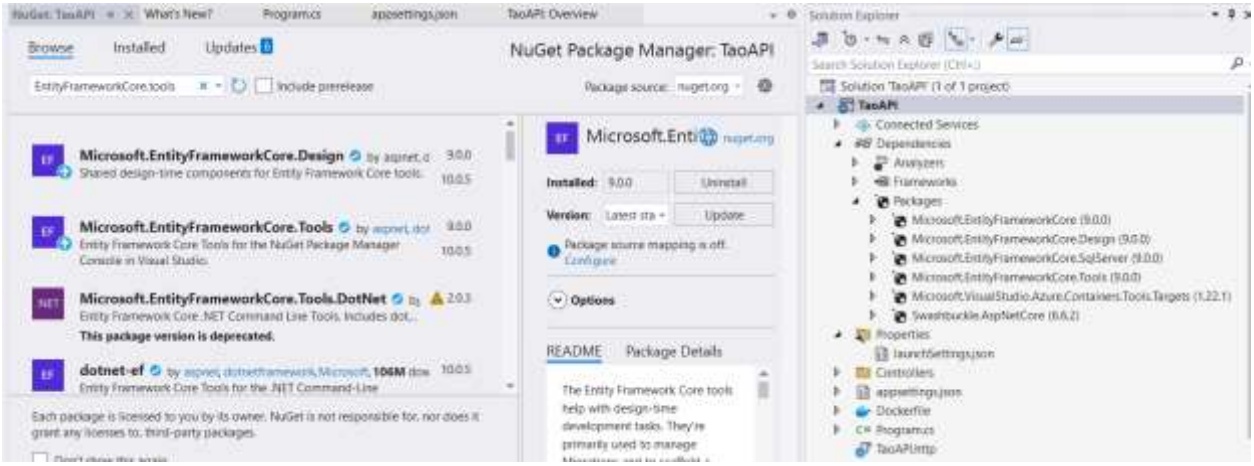
➢ Thiết kế cơ sở dữ liệu (ApiDemo) trong SQL Server với 2 bảng

- tblLoaiSP(MaLoai, TenLoai)
- tblSanPham(MaSP, TenSP, SoLuong, MaLoai)



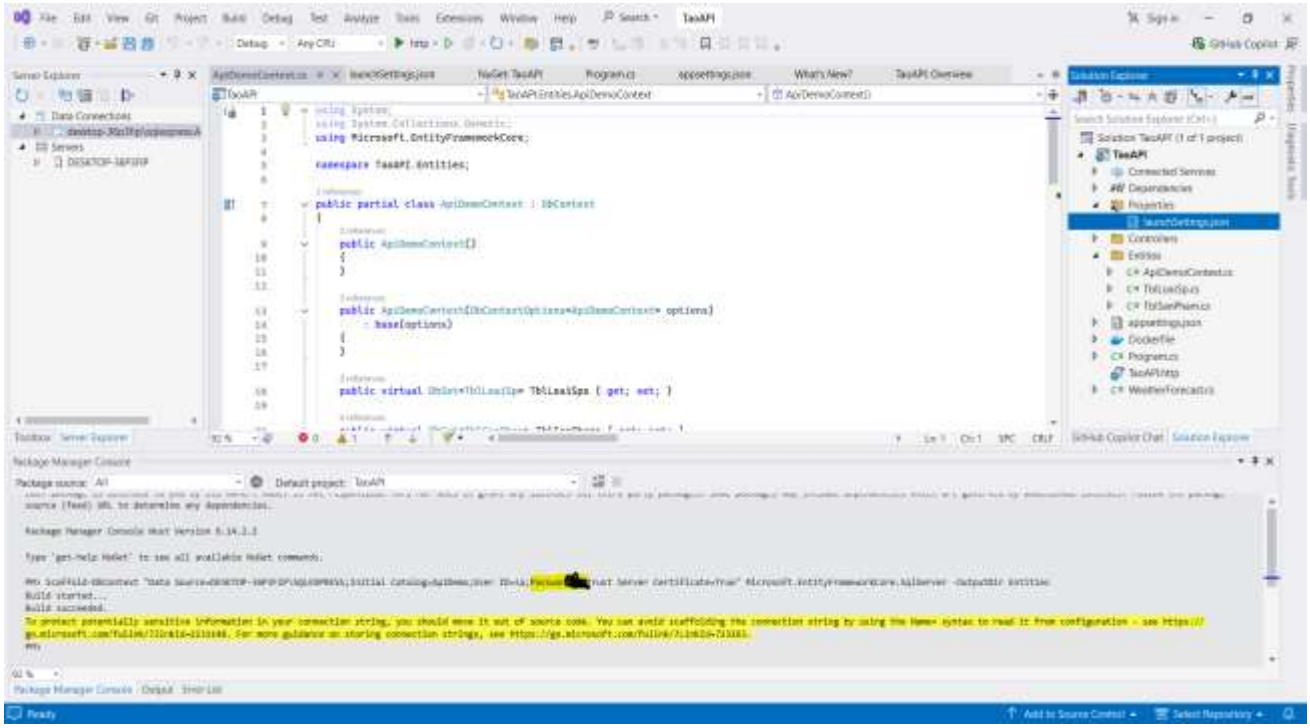
➢ Cài đặt các gói EntityFramworkCore để có thể kết nối database (chú ý lựa chọn version phù hợp với framework đang sử dụng)

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Design



➢ Sử dụng EntityFramework để liên kết dữ liệu: sử dụng câu lệnh: Scaffold-DbContext "ConnectionString" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Folder

- ConnectString là chuỗi kết nối tới CSDL. Ví dụ: *Data Source=DESKTOP-36P3FIP\SQLEXPRESS;Initial Catalog=ApiDemo;User ID=sa;Password=***;Trust Server Certificate=True*
- Folder: là thư mục lưu trữ kết quả. Ví dụ: Entities



➢ Kiểm tra lại địa chỉ của API đã xây dựng trong file launchSettings.json

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5130"
    },
    "IIS Express": {
```

Như vậy, địa chỉ của API vừa tạo là http://localhost:5130

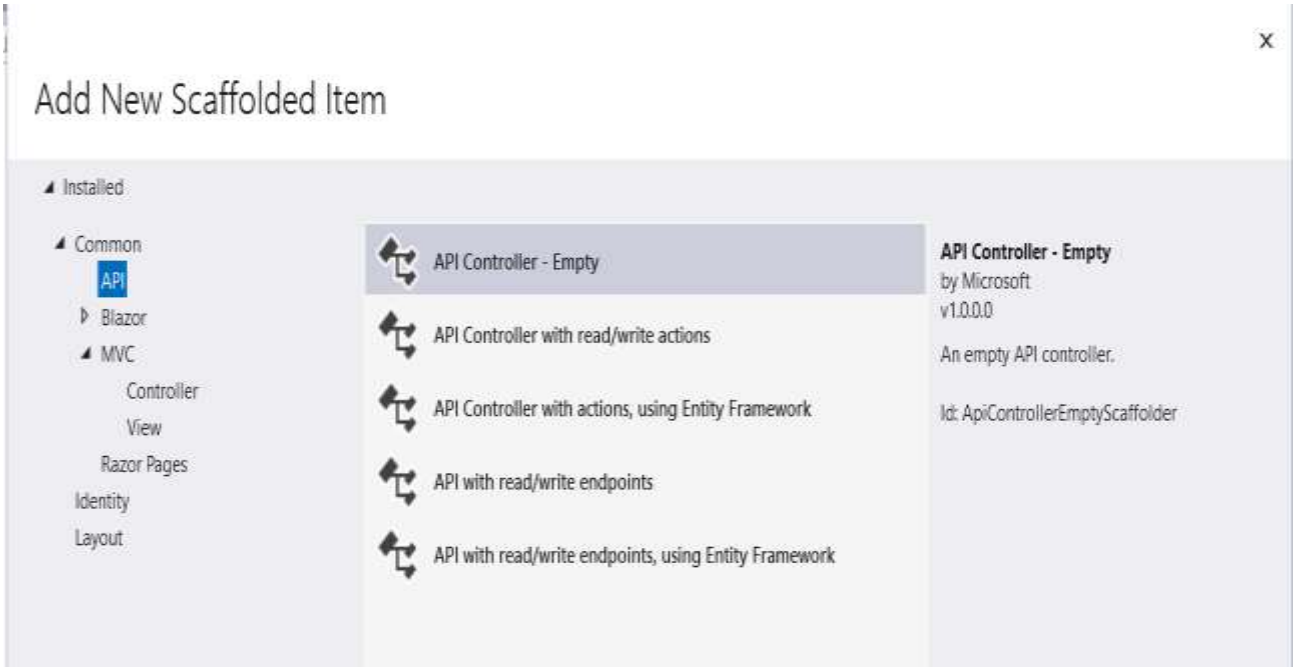
➢ Lưu ý: khi kết nối với API ta cần lưu ý như sau:

- Máy ảo android: thay localhost bằng 10.0.2.2 (http://10.0.2.2:5130)
- Máy ảo IOS: Có thể dùng localhost hoặc 127.0.0.1(http://localhost:5130)
- Thiết bị thật: Phải dùng địa chỉ IP LAN của máy tính

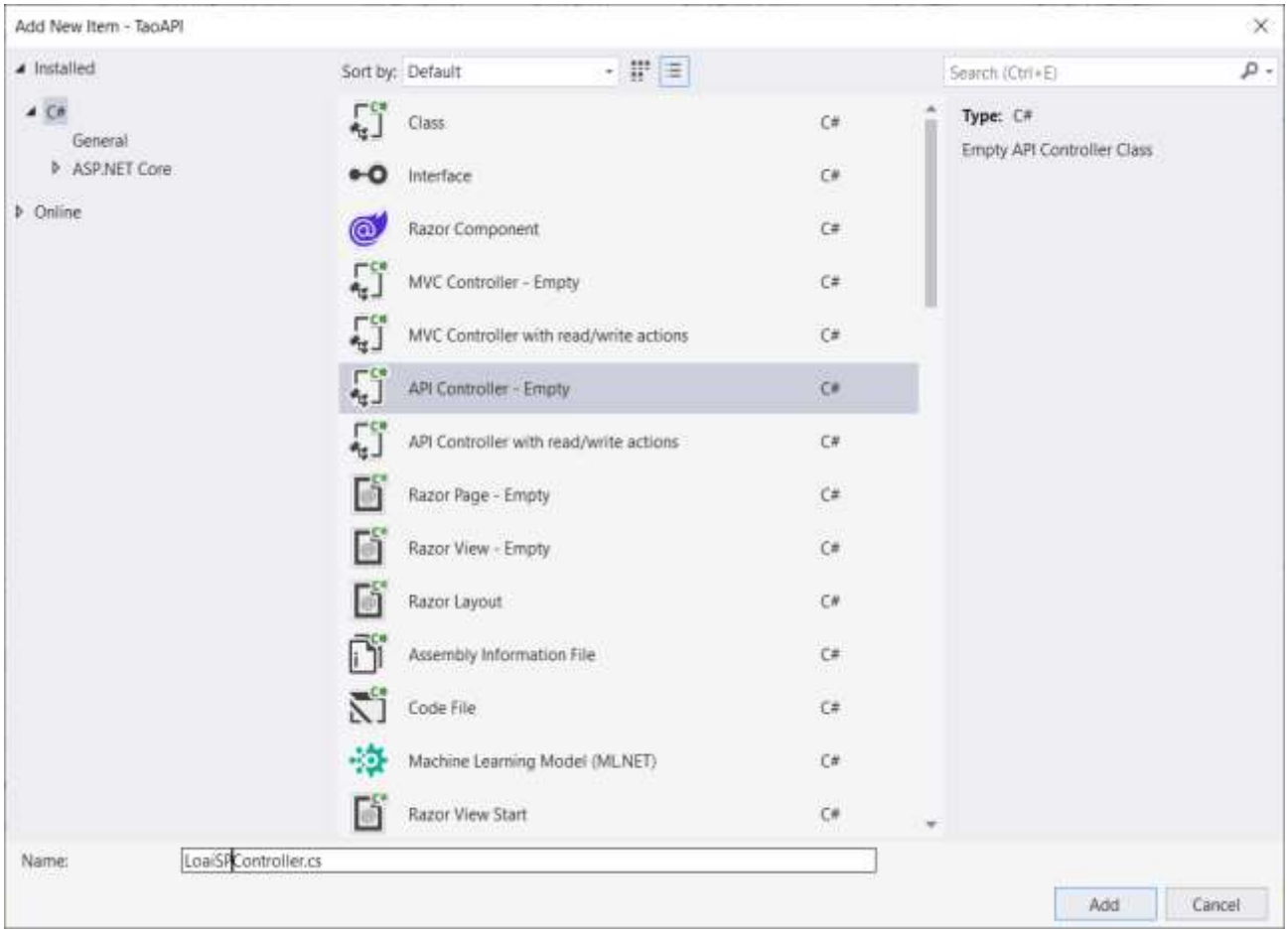
c. Xây dựng các API

➢ Xây dựng API Controller

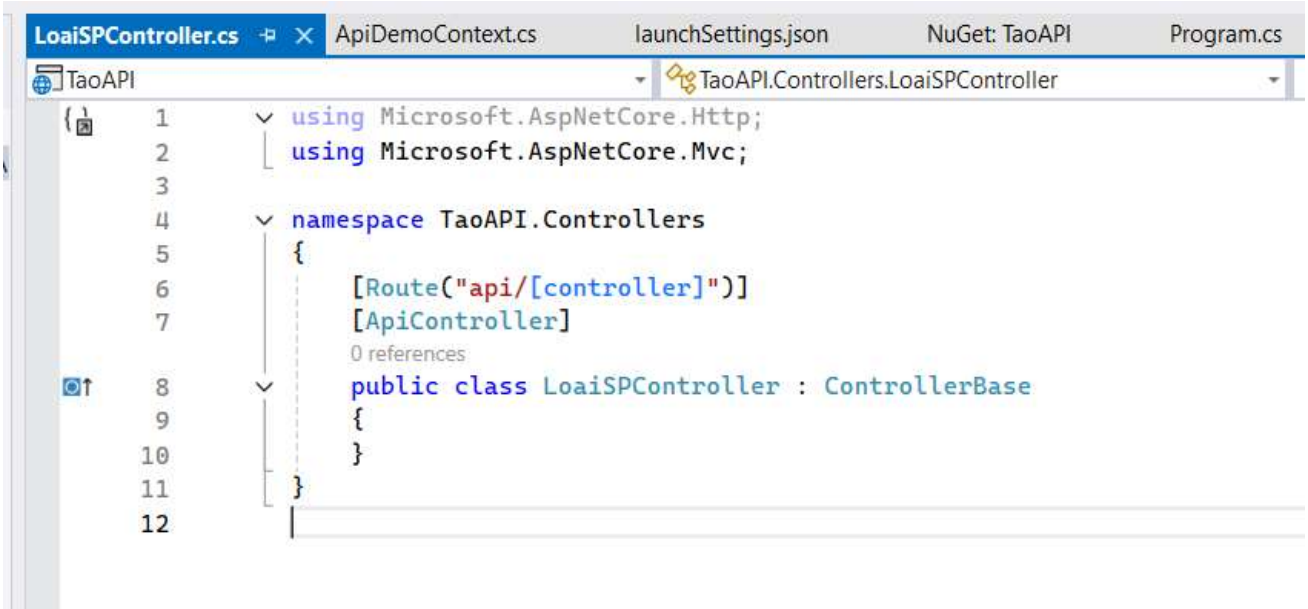
- Thêm một Controller vào project



- Đặt tên Controller



- Kết quả thu được:



➢ Cấu hình chuỗi kết nối và thực thi API

- Khai báo chuỗi kết nối vào file appsettings.json



- Khai báo dependencies cho Dbcontext trong program.cs

```
builder.Services.AddDbContext<ApiDemoContext>(option =>
    option.UseSqlServer(builder.Configuration.GetConnectionString("ApiDemo")));
```

- Khai báo dependencies cho phép flutter truy xuất vào API

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});
```

Và

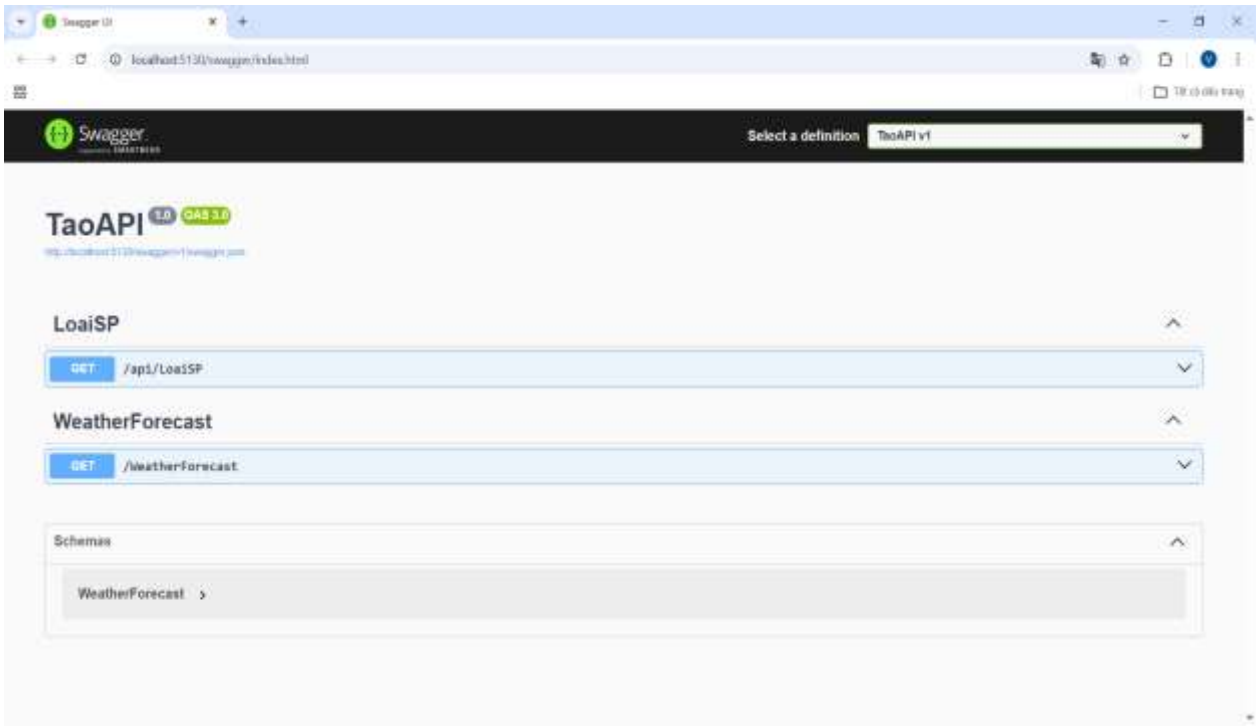
```
app.UseCors("AllowAll");
```

- Xây dựng API lấy dữ liệu (lấy thông tin của các loại SP): đây là dạng API với phương thức GET để lấy dữ liệu. Do đó, ta cần phả khai báo thêm DBContext

```
namespace TaoAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class LoaiSPController : ControllerBase
    {
        private readonly ApiDemoContext _con;

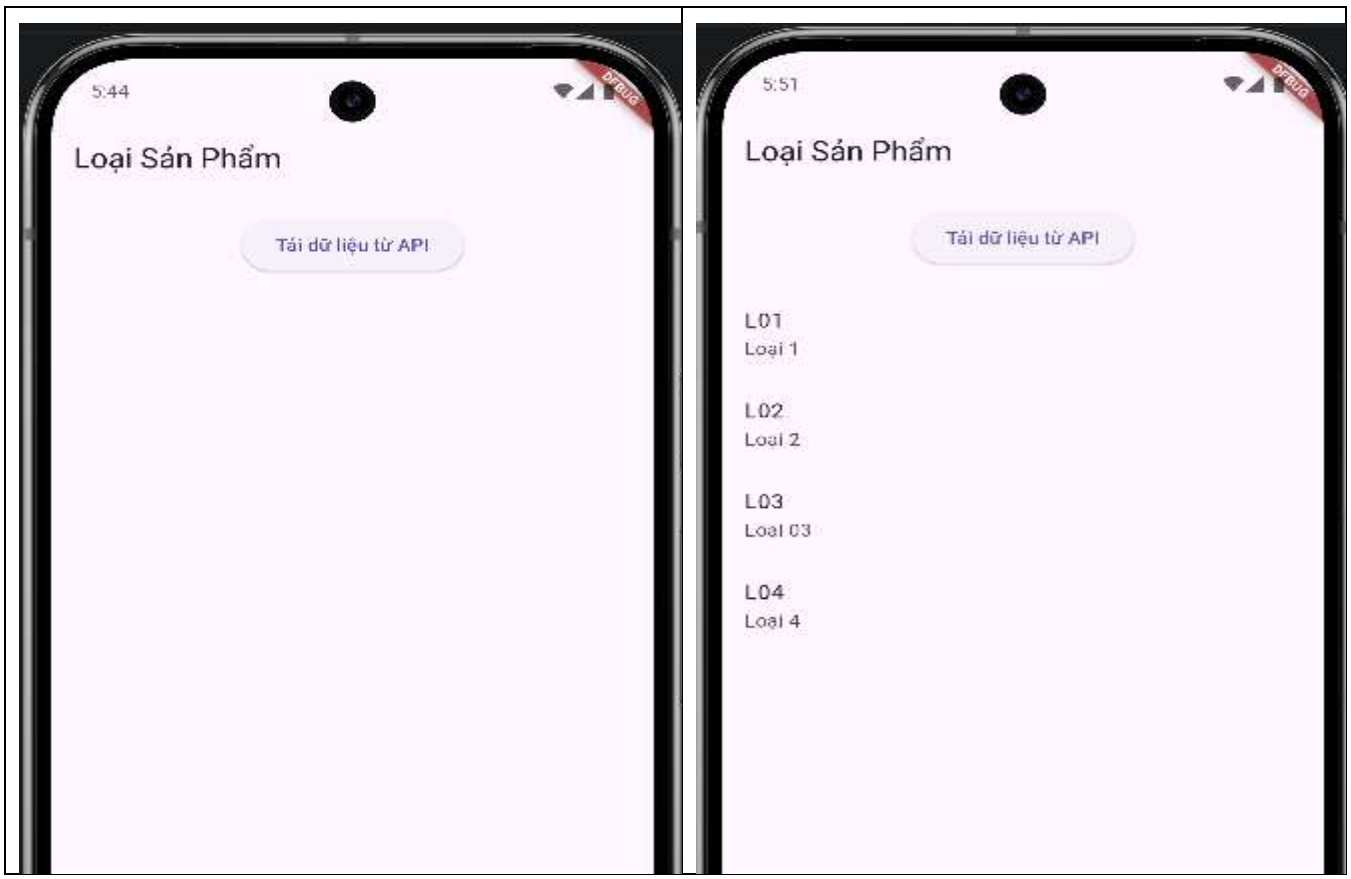
        0 references
        public LoaiSPController(ApiDemoContext context)
        {
            _con = context;
        }
        [HttpGet]
        0 references
        public IActionResult GetAll()
        {
            return Ok(_con.TblLoaiSps.ToList());
        }
    }
}
```

- Thực thi va xem kết quả đạt được (chạy với tùy chọn http)



d. Kết nối flutter

Xây dựng ứng dụng đơn giản mình họa việc kết nối flutter với API vừa xây dựng với kết quả được minh họa như sau



➢ Xây dựng lớp LoaiSP

```
class LoaiSP {
  String maLoai;
  String tenLoai;
  LoaiSP({required this.maLoai, required this.tenLoai});

  factory LoaiSP.fromJson(Map<String, dynamic> json) {
    return LoaiSP(
      maLoai: json['maLoai'],
      tenLoai: json['tenLoai']);
  }
}
```

➢ Xây dựng giao diện

```
class TestAPIDemo extends StatelessWidget {
  const TestAPIDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: LoadLoaiSP(title: 'Loại Sản Phẩm'),
    );
  }
}
```

```dart
class LoadLoaiSP extends StatefulWidget {
  const LoadLoaiSP({super.key, required this.title});
  final String title;

  @override
  State<LoadLoaiSP> createState() => _LoadLoaiSPState();
}

class _LoadLoaiSPState extends State<LoadLoaiSP> {
  List<LoaiSP> loaiSPs = [];
  bool isLoading = false;

  static const String baseUrl = 'http://10.0.2.2:5130/api/LoaiSP';
  // Hàm gọi API
  Future<void> fetchData() async {
    setState(() {
      isLoading = true;
    });

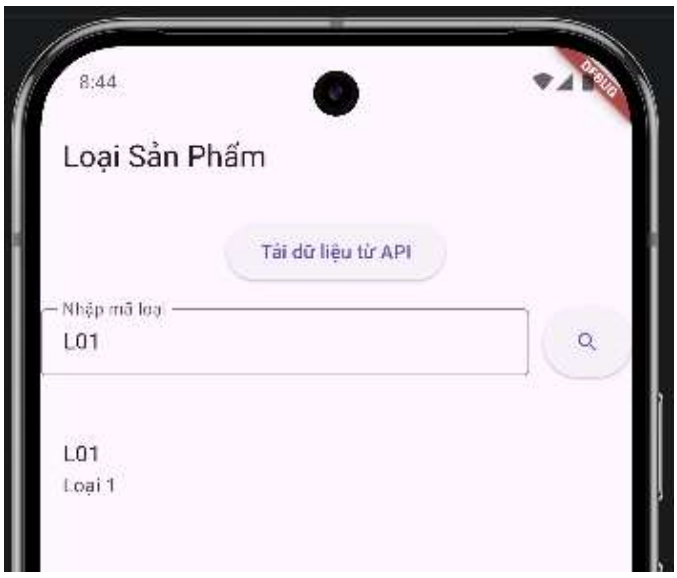
    try {
      // Lưu ý: Nếu chạy trên Emulator Android, sử dụng 10.0.2.2
      final response = await http.get(Uri.parse(baseUrl));
     // Lấy dữ liệu thành công
      if (response.statusCode == 200) {
        setState(() {
          loaiSPs = (jsonDecode(response.body) as List)
              .map((e) => LoaiSP.fromJson(e))
              .toList();
          isLoading = false;
        });
      } else {
        throw Exception('Lỗi tải dữ liệu: ${response.body}  ');
      }
    } catch (e) {
      setState(() {
        isLoading = false;
      });
      // Hiển thị thông báo lỗi nếu không kết nối được
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Không thể kết nối API')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(widget.title)),
      body: Column(
        children: [
          Padding(
```

```
            padding: const EdgeInsets.all(16.0),
            child: ElevatedButton(
              onPressed: isLoading ? null : fetchData,
              child: isLoading
                  ? const SizedBox(
                      width: 20,
                      height: 20,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Tải dữ liệu từ API'),
            ),
          ),
          Expanded(
            child: ListView.builder(
              itemCount: loaiSPs.length,
              itemBuilder: (context, index) {
                return ListTile(
                  title: Text(loaiSPs[index].maLoai),
                  subtitle: Text(loaiSPs[index].tenLoai),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}
```

2. Bài tập hướng dẫn

Ví dụ 1: Phát triển ứng dụng để có thể tìm loại sản phẩm theo mã



Hướng dẫn:

Bổ sung thêm phần giao diện

```
    Row(
            children: [
              // Phải có Expanded để TextField không chiếm vô hạn chiều ngang
              Expanded(
                child: TextField(
                  controller: _controller,
                  decoration: const InputDecoration(
                    labelText: 'Nhập mã loại',
                    border: OutlineInputBorder(),
                    isDense: true, // Làm cho ô nhập gọn hơn để vừa hàng
                  ),
                ),
              ),
              const SizedBox(width: 10), // Khoảng cách giữa ô nhập và nút

              ElevatedButton(
                onPressed: isLoading ? null : () =>
searchLoaiSP(_controller.text),
                style: ElevatedButton.styleFrom(
                  padding: const EdgeInsets.symmetric(
                    vertical: 16,
                    horizontal: 20,
                  ),
                ),
                child: isLoading
                    ? const SizedBox(
                        width: 20,
                        height: 20,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      )
                    : const Icon(Icons.search),
              ),
            ],
          ),
  const Divider(height: 30),
```

Cài đặt các hàm xử lý

```
  final TextEditingController _controller = TextEditingController();
   Future<void> searchLoaiSP(String maLoai) async {
    if (maLoai.isEmpty) return;
    setState(() => isLoading = true);
    // Lưu ý: Đổi 10.0.2.2 thành localhost nếu dùng iOS Simulator
    final url = Uri.parse('http://10.0.2.2:5130/api/LoaiSP/$maLoai');
    try {
      final response = await http.get(url);
      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        setState(() {
          loaiSPs = [LoaiSP.fromJson(data)];
          isLoading = false;
```

```
        });
      } else {
        setState(() {
          loaiSPs = [];
          isLoading = false;
        });
        showSnackBar("Không tìm thấy loại SP này!");
      }
    } catch (e) {
      setState(() => isLoading = false);
      showSnackBar("Lỗi kết nối: $e");
    }
  }

  void showSnackBar(String message) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
  }
```

Ví dụ 2: Xây dựng thêm loại sản phẩm vào cơ sở dữ liệu.



Hướng dẫn :

Xây dựng lớp LoaiSPModel trên NetCore

```
public class LoaiSPModel
{
    [Required]
    [MaxLength(10)]
    public string LoaiId { get; set; } = string.Empty;
    [Required]
    [MaxLength(50)]
    public string LoaiName { get; set; } = string.Empty;
}
```

Thêm API post vào controller

```
[HttpPost]
public IActionResult Create(LoaiSPModel loaiModel)
{
    try
    {
        var loai = new TblLoaiSp
        {
            MaLoai = loaiModel.LoaiId,
            TenLoai = loaiModel.LoaiName
        };

        //_con.Add(loai);
        _con.TblLoaiSps.Add(loai);
        _con.SaveChanges();
        return Ok(loai);
    }
    catch (Exception ex)
    {
        return BadRequest();
    }
}
```

Xử lý thêm loại sản phẩm trên flutter

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class AddLoaiSPPage extends StatefulWidget {
  const AddLoaiSPPage({super.key});

  @override
  State<AddLoaiSPPage> createState() => _AddLoaiSPPageState();
}

class _AddLoaiSPPageState extends State<AddLoaiSPPage> {
  // Các controller để lấy dữ liệu từ Form
  final TextEditingController _maLoaiController = TextEditingController();
  final TextEditingController _tenLoaiController = TextEditingController();
  bool _isSending = false;

  // Hàm thực hiện gửi dữ liệu lên API
  Future<void> saveLoaiSP() async {
    final maLoai = _maLoaiController.text.trim();
    final tenLoai = _tenLoaiController.text.trim();

    if (maLoai.isEmpty || tenLoai.isEmpty) {
      _showSnackBar("Vui lòng nhập đầy đủ thông tin!");
      return;
    }
    setState(() => _isSending = true);
    final url = Uri.parse('http://10.0.2.2:5130/api/LoaiSP');
    try {
      final response = await http.post(
```

```
      url,
      headers: {"Content-Type": "application/json"},
      body: jsonEncode({
        "LoaiId": _maLoaiController.text.trim(),
        "LoaiName": _tenLoaiController.text.trim(),
      }),
    );

    if (response.statusCode == 201 || response.statusCode == 200) {
      _showSnackBar("Thêm thành công!");
      _maLoaiController.clear();
      _tenLoaiController.clear();
    } else {
      _showSnackBar("Lỗi từ server: ${response.statusCode}");
    }
  } catch (e) {
    _showSnackBar("Lỗi kết nối: $e");
  } finally {
    setState(() => _isSending = false);
  }
}

void _showSnackBar(String message) {
  ScaffoldMessenger.of(context,
  ).showSnackBar(SnackBar(content: Text(message)));
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Thêm Loại Sản Phẩm Mới')),
    body: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text("Mã Loại SP:",
            style: TextStyle(fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 8),
          TextField(
            controller: _maLoaiController,
            decoration: const InputDecoration(
              hintText: "Nhập mã (ví dụ: L05)",
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 20),
          const Text("Tên Loại SP:",
            style: TextStyle(fontWeight: FontWeight.bold),
          ),
```

```
            const SizedBox(height: 8),
            TextField(
              controller: _tenLoaiController,
              decoration: const InputDecoration(
                hintText: "Nhập tên loại sản phẩm",
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 30),
            SizedBox(
              width: double.infinity,
              height: 50,
              child: ElevatedButton(
                onPressed: _isSending ? null : saveLoaiSP,
                style: ElevatedButton.styleFrom(backgroundColor: Colors.blue),
                child: _isSending
                    ? const CircularProgressIndicator(color: Colors.white)
                    : const Text("LƯU VÀO DATABASE",
                        style: TextStyle(color: Colors.white),
                      ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Ví dụ 3: Chỉnh sửa lại ví dụ 1 để trên Listview hiển thị loại sản phẩm có thêm IconButton xóa loại sản phẩm. Khi bấm vào button xóa, loại sản phẩm đó sẽ bị xóa khỏi cơ sở dữ liệu



Hướng dẫn :

Xây dựng API xóa một loại sản phẩm trong cơ sở dữ liệu

```
[HttpDelete("{id}")] // Tham số {id} trên URL sẽ được truyền vào biến string id
public IActionResult Delete(string id)
{
    try
    {
        var loai = _con.TblLoaiSps.FirstOrDefault(x => x.MaLoai == id);
        if (loai == null)
        {
            return NotFound(new { message = "Không có mã SP: " + id });
        }
        _con.TblLoaiSps.Remove(loai);
        _con.SaveChanges();
        return Ok(new { message = "Xóa thành công loại SP mã: " + id });
    }
    catch (Exception ex)
    {
        // Lỗi loại SP đang có các sản phẩm con (ràng buộc khóa ngoại)
        return BadRequest(new
        {
            message = "Không thể xóa loại sản phẩm này.",
            error = ex.Message
        });
    }
}
```

Chỉnh sửa lại phần hiển thị của ListView

```
ListTile(
       title: Text(item.tenLoai),
       subtitle: Text(item.maLoai),
       trailing: IconButton(
            icon: const Icon(Icons.delete, color: Colors.red),
                  onPressed: () {
                  // Gọi hàm xóa khi người dùng bấm nút
                 deleteLoaiSP(item.maLoai);
              },
       ),
)
```

Xây dựng phương thức xóa loại sản phẩm

```
Future<void> deleteLoaiSP(String maLoai) async {
  final url = Uri.parse('http://10.0.2.2:5130/api/LoaiSP/$maLoai');
  try {
    final response = await http.delete(url);
    if (response.statusCode == 200) {
      // Thông báo thành công
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text("Đã xóa mã $maLoai thành công!")),
      );
      // để cập nhật lại giao diện (ListView)
      fetchData();
    } else {
      print("Lỗi: ${response.statusCode}");
```

```
    }
  } catch (e) {
    print("Lỗi kết nối: $e");
  }
}
```

Bổ sung : API Cập nhật loại sản phẩm

```
[HttpPut]
public IActionResult Update(LoaiSPModel loaiModel)
{
    try
    {
        // 1. Tìm đối tượng cũ trong Database
        var loai = _con.TblLoaiSps.FirstOrDefault(x => x.MaLoai ==
                 loaiModel.LoaiId);
        if (loai == null)
        {
            return NotFound(new { message = "Không có: " + loaiModel.LoaiId });
        }
        loai.TenLoai = loaiModel.LoaiName;
        _con.TblLoaiSps.Update(loai);
        _con.SaveChanges();
        return Ok(loai);
    }
    catch (Exception ex)
    {
        return BadRequest(new { message = "Thất bại", error = ex.Message });
    }
}
```

Bổ sung hàm cập nhật dữ liệu trong flutter

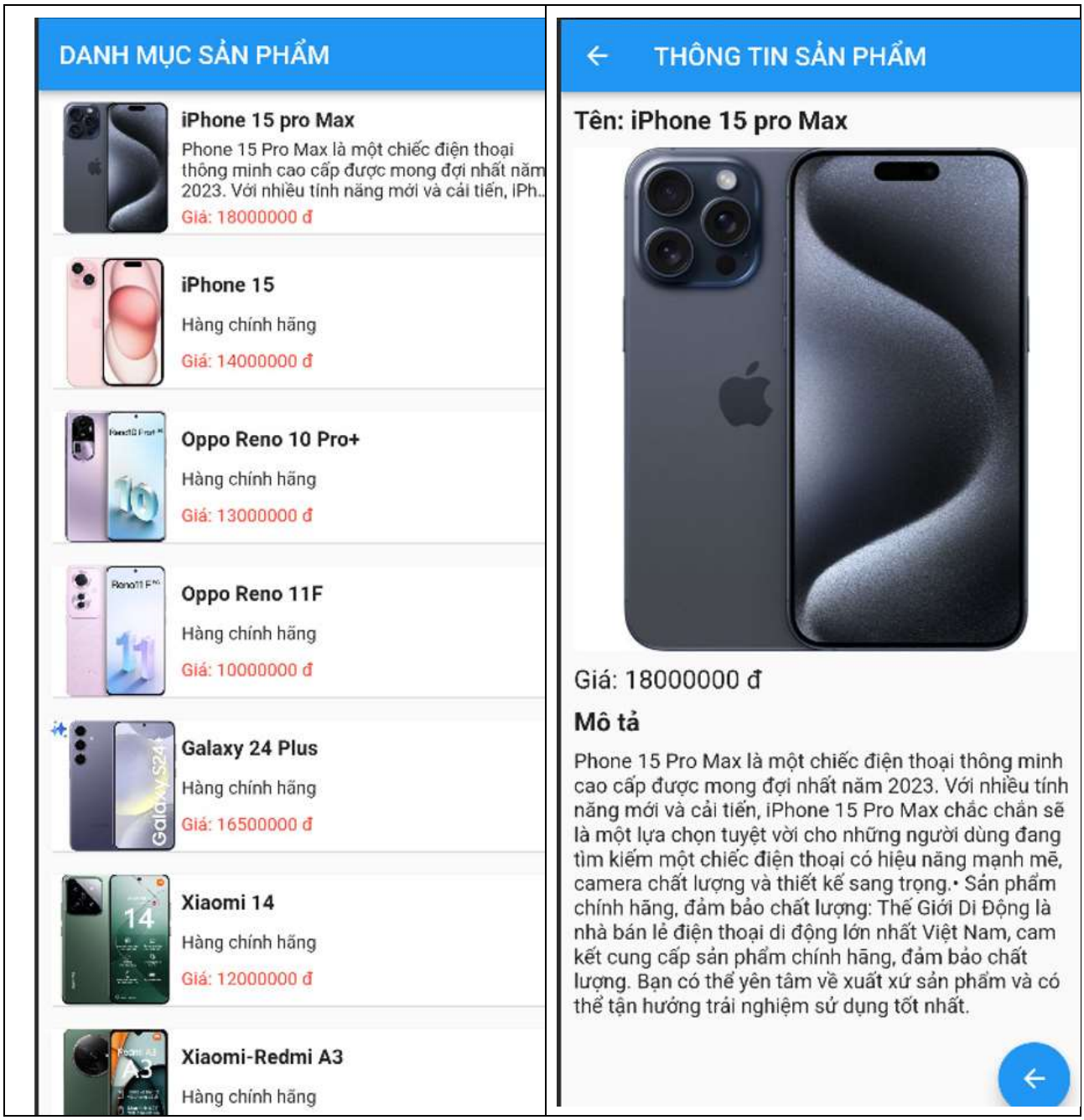
```
Future<void> updateLoaiSP(String maLoai, String tenMoi) async {
    final url = Uri.parse('http://10.0.2.2:5130/api/LoaiSP/');

    try {
      final response = await http.put(url,
        headers: {"Content-Type": "application/json"},
        body: jsonEncode({
          "loaiId": maLoai, // Khớp với Model của bạn
          "loaiName": tenMoi,
        }),
      );

      if (response.statusCode == 200) {
        _showSnackBar("Cập nhật thành công!");
        Navigator.pop(context); // Quay lại trang danh sách
      } else {
        _showSnackBar("Lỗi cập nhật!");
      }
    } catch (e) {
      print("Lỗi: $e");
    }
  }
```

3. Bài tập tại lớp :

Bài tập 01. Thiết kế CSDL và xây dựng giao diện hiển thị thông tin của các điện thoại trong flutter.



Bài tập 02. Làm lại bài tập về hiển thị thông tin của các loại đồ án (đã làm trong bài thực hành 3) nhưng lấy dữ liệu được thiết kế trong SQL server



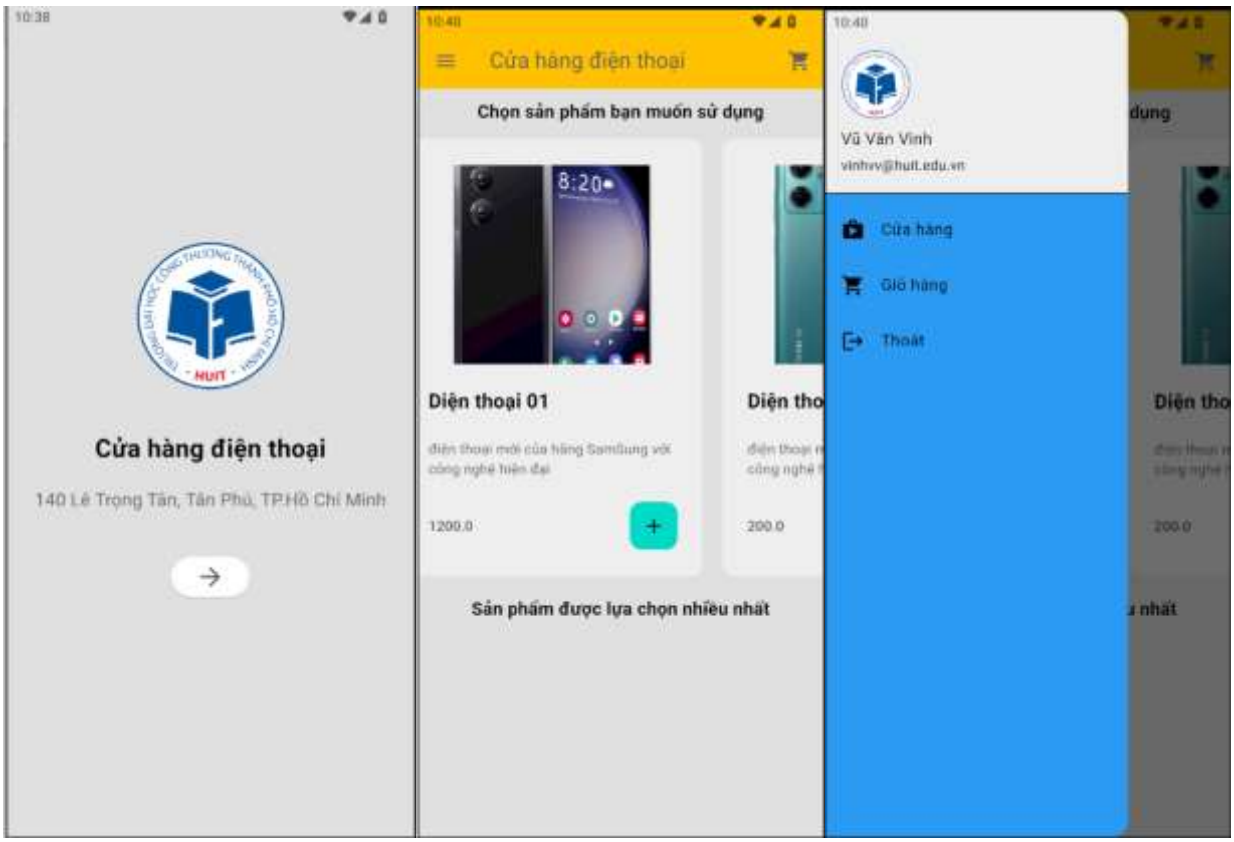
4. Bài tập về nhà :

Bài tập 03. Sinh viên thiết kế cơ sở dữ liệu trên SQL Server và xây dựng ứng dụng mua bán điện thoại di động đơn giản sau. Lưu ý lưu lại các hóa đơn của khách hàng sau khi mau

Mô tả : Khi chương trình được thực thi, màn hình giới thiệu ứng dụng được hiện lên như hình. Người dùng bấm vào button mũi tên để vào ứng dụng, màn hình hiển thị sản phẩm.

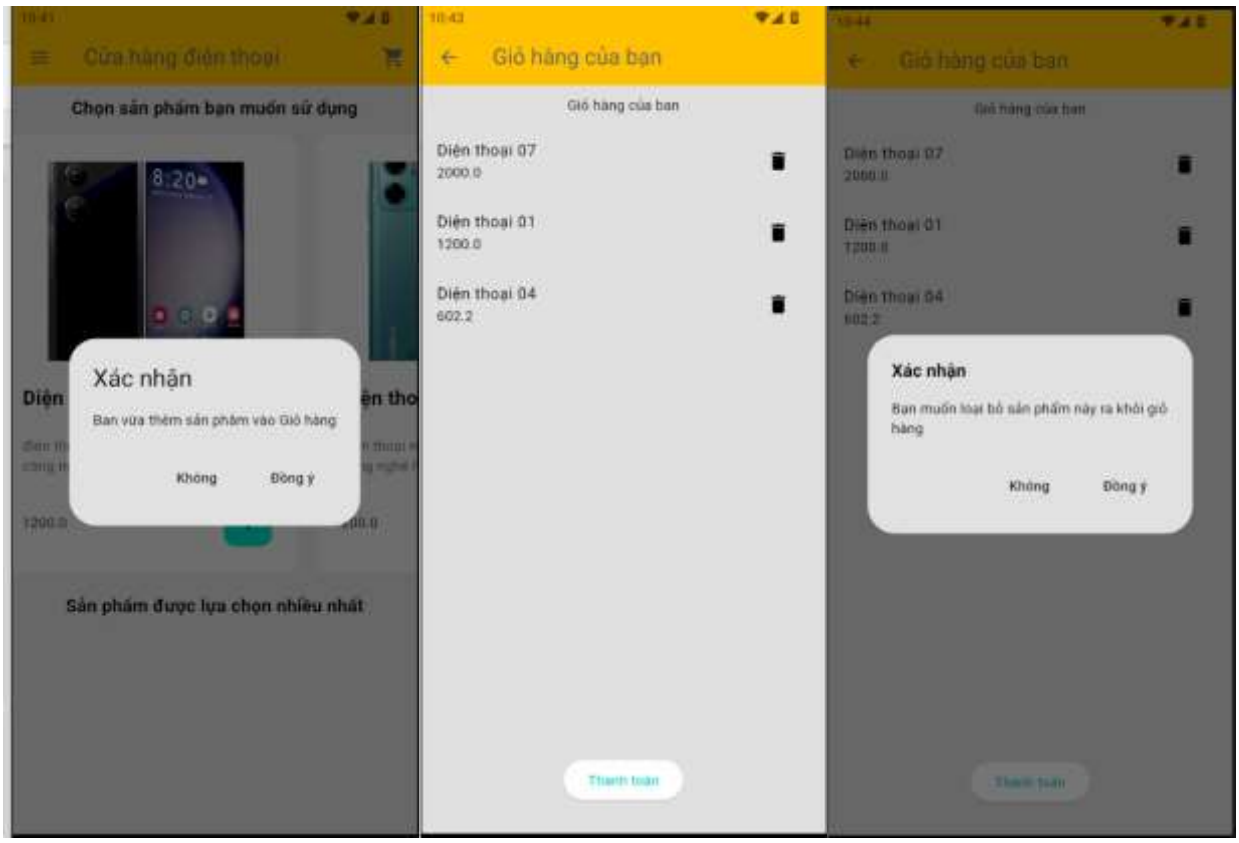
Màn hình ứng dụng hiển thị sản phẩm sẽ hiển thị điện thoại mà cửa hàng đang bán với các thông tin như màn hình. Trong màn hình sẽ có thiết kế Drawer như trong hình minh họa

- Giỏ hàng : hiển thị thông tin giỏ hàng
- Cửa hàng : là trang hiển thị sản phẩm
- Thoát : là trang giới thiệu



Khi khác hàng muôn mua sản phẩm nào thì bấm vào button. Khi khách hàng chọn mua một sản phẩm thì sẽ có hội thoại xác nhận. Khách hàng có thể chọn nhiều sản phẩm. Khi khách hàng hoàn thành việc mua hàng, khác hàng có thể bấm vào biểu tượng giỏ hàng trên thanh công cụ hoặc bấm vào Giỏ hàng trong Drawer.

Màn hình giỏ hàng sẽ hiển thị thông tin của các sản phẩm mà khách hàng đã chọn. Nếu khách hàng chưa chọn sản phẩm nào thì sẽ thông báo là « Bạn chưa có sản phẩm nào trong giỏ hàng ». Với mỗi sản phẩm trong giỏ hàng, khách hàng có thể bấm vào biểu tượng để loại bỏ sản phẩm đó. Trước khi loại bỏ, khác hàng sẽ nhận được thông báo xác nhận như hình bên dưới



Trong màn hình giỏ hàng, người dùng có thể bấm vào button thanh toán để xác nhận thanh toán hoặc trở lại trang sản phẩm để mua thêm sản phẩm.

